Systèmes d'Exploitation



Objectif :

Comprendre comment un système d’exploitation fonctionne et comment l’utiliser

Sami Radi
sami.radi@virtuoworks.com



Permissions avancées

(cours précédent : Gestion des Paquets)

Sami Radi
sami.radi@virtuoworks.com

I. Systèmes d'Exploitation
I.12. Permissions avancées



I. Systèmes d'Exploitation

1. Sommaire & Introduction
2. Noyau et Pilotes
3. Utilisateurs et Sessions
4. Système de Fichier
5. Permissions et Droits
6. Shell et Utilitaires
7. Gestion de la Mémoire
8. Programmes et Processus
9. Variables d'Environnement
10. Scripts Shell
11. Gestion des Paquets
12. Permissions avancées
13. Gestionnaire de services et Journalisation
14. Tâches planifiées

Sami Radi
sami.radi@virtuoworks.com



Permissions avancées

Sami Radi
sami.radi@virtuoworks.com

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.a Permissions par défaut



- Lorsqu'un fichier est créé, le système propose des **permissions par défaut** :
  - Fichier : `644` (lecture/écriture pour l'utilisateur, lecture pour le groupe et les autres)
  - Répertoire : `755` (lecture/écriture/exécution l'utilisateur, lecture/exécution pour les autres)

- La commande `umask` permet de définir les **permissions à retirer** lors de la création d'un fichier ou répertoire. **Elle ne donne pas des permissions**, mais retire celles qui seraient accordées par défaut.

Sami Radi
sami.radi@virtuoworks.com

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.a Permissions par défaut



- Fonctionnement :
    - Le umask est soustrait :
        - 666 (110 110 110) - 022 (000 010 010) = 644 (110 100 100)
          → fichier avec `rw-r--r--`
        - 777 (111 111 111) - 022 (000 010 010) = 755 (111 101 101)
          → répertoire avec `rwxr-xr-x`

- Exemple :

```
umask 027
touch toto.txt
ls -l toto.txt
```

- 644 (permissions par défaut) - 027 (masque) = 640

→ Le fichier aura les permissions `rw-r-----`

Sami Radi
sami.radi@virtuoworks.com

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- **`setuid`** (**`s`** sur utilisateur)
  - Appliqué à un fichier exécutable.
  - Lors de l'exécution, le processus prend les **droits du propriétaire** du fichier, et non de l'utilisateur qui l'exécute.

- Exemple :
  - `-rwsr-xr-x 1 root root 12345 oct 20 /usr/bin/passwd`

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- Exemple concret :
  - Prenons un programme `changer_mdp` appartenant à root, qui modifie un fichier système comme `/etc/shadow`.

- Ses permissions sont :
  - `-rwxr-xr-x 1 root root 12345 oct 20 /usr/bin/changer_mdp`

- Application du setuid :
  - `chmod u+s /usr/bin/changer_mdp`

- Désormais :
  - `-rwsr-xr-x 1 root root 12345 oct 20 /usr/bin/changer_mdp`
  - Le `s` dans `rws` indique que le bit `setuid` est actif.
  - Un utilisateur lambda peut exécuter ce programme avec **les droits root**, sans être root lui-même.

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- **`setgid`** (**`s`** sur groupe)
  - Sur un fichier exécutable : le processus hérite du **groupe** du fichier.
  - Sur un répertoire : les fichiers créés héritent du **groupe du répertoire**, pas de l'utilisateur.

- Exemple :
  - `drwxrwsr-x 2 user devs 4096 oct 20 /projets`

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- Exemple concret :
  - Un répertoire `/projets` partagé entre plusieurs utilisateurs du groupe `devs`. Tous les fichiers dans `/projets` doivent appartenir au groupe `devs`

- Ses permissions sont :
  - `drwxrwxr-x 2 alice devs 4096 oct 20 /projets`
  - Par défaut tout ce que je créé dans `/projets` appartient à **mon groupe principal**

- Application du `setgid` :
  - `chmod g+s /projets`

- Désormais :
  - `drwxrwsr-x 2 alice devs 4096 oct 20 /projets`

  - Le `s` dans `rws` sur le groupe indique que le bit `setgid` est actif.
  - Tous les fichiers créés dans `/projets` auront le groupe `devs`.

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- Sticky Bit (`t`) :
  - Appliqué aux répertoires partagés (ex : `/tmp`)
  - **Empêche** un utilisateur de supprimer un fichier qu'il ne possède pas, même s'il a les droits d'écriture sur le répertoire.

- Exemple :
  - `drwxrwxrwt 10 root root 4096 oct 20 /tmp`

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.b Permissions spéciales



- Exemple concret :
  - Le répertoire `/partage` est accessible en écriture par tous, mais chaque utilisateur ne doit pouvoir supprimer que ses propres fichiers. Ici tous les utilisateurs peuvent supprimer n'importe quel fichier :
  - `drwxrwxrwx 10 root etudiants 4096 oct 20 /partage`

Application du sticky bit :

- `chmod +t /partage`

- Desormais :
  - `drwxrwxrwt 10 root etudiants 4096 oct 20 /partage`
  - Cela protège les fichiers dans `/partage` contre la suppression par des utilisateurs non propriétaires.

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.c ACL (Access Control Lists)



- Les ACL permettent de dépasser le modèle classique pour `u`, `g` et `o` en attribuant des permissions **granulaires** à plusieurs utilisateurs ou groupes.

- Fonctionnement :

  - Chaque fichier peut avoir une **liste de contrôle d'accès**.
  - Les ACL sont stockées en métadonnées du système de fichiers.

- Commandes :

  - `getfacl` : affiche les ACL

  - `setfacl` : modifie les ACL

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.c ACL (Access Control Lists)



- Exemple :
- On a un fichier `rapport.txt` appartenant à l'utilisateur `alice` :
  - `ls -l rapport.txt`
  - `-rw------- 1 alice alice 1024 oct 20 rapport.txt`

- Alice veut que l'utilisateur bob puisse le lire, **sans modifier les permissions classiques** ni changer le groupe :

  - `setfacl -m u:bob:r rapport.txt`
    - `-m` : modifier
    - `u:bob:r` : donne à l'utilisateur bob le droit de lecture (r)

- Vérifier l'application de l'ACL :
  - `getfacl rapport.txt`

→ Bob a des droits spécifiques sur `rapport.txt`

I. Systèmes d'Exploitation
I.12. Permissions avancées
I.12.c ACL (Access Control Lists)



- Types d'entrées ACL :
  - `user::` → propriétaire
  - `user:alice:` → utilisateur spécifique
  - `group::` → groupe propriétaire
  - `group:etudiants:` → groupe spécifique
  - `mask::` → limite maximale des permissions effectives
  - `other::` → autres utilisateurs

- Attention au masque ACL :
  - Le `mask` définit les permissions maximales que peuvent avoir les entrées `user:` et `group:`
  - Si le `mask` est plus restrictif, il limite les droits même si l'entrée ACL est plus permissive.



Gestionnaire des services et Journalisation

(voir cours suivant : Systèmes d'Exploitation – Gestionnaire des services et Journalisation)

Sami Radi
sami.radi@virtuoworks.com